

LABORATORIO DI INFORMATICA

Modulo M1

Markdown, documentazione e ambiente Jupyter

Dispensa

Istituto Tecnico Tecnologico "G. Marconi" — Rovereto

Indirizzo Informatica e Telecomunicazioni, articolazione Informatica

Classe 4Bi — Anno scolastico 2026-2027 · Docente di laboratorio: Andrea Trentini

Modulo M1 — Markdown, documentazione e ambiente Jupyter

Voce	Contenuto
Periodo	21 settembre - 9 ottobre 2026 (settimane 2-5)
Ore	12, di cui 8 su Markdown e documentazione tecnica e 4 sull'ambiente Jupyter
Prerequisiti	Uso individuale di Git: <code>add</code> , <code>commit</code> , <code>push</code> , lettura della cronologia. Editor Visual Studio Code installato. Uso essenziale del terminale di Windows. Conoscenza di Java per gli esempi di codice documentato. Primo accesso al Jupyter server d'istituto, già effettuato nel modulo M0.
Obiettivi	Conoscere la sintassi Markdown e le differenze pratiche fra le sue varianti; redigere un <code>README.md</code> completo per un progetto proprio; descrivere l'architettura di un programma con diagrammi Mermaid; convertire un documento in HTML e in PDF; applicare le convenzioni della scrittura tecnica; usare il Jupyter server d'istituto sapendo che cosa fa ciascuno dei tre componenti dell'ambiente e perché l'ordine di esecuzione delle celle determina il risultato.
Prodotto finale	Il <code>README.md</code> del repository personale del corso e un primo notebook eseguito e salvato sul server d'istituto.

Questo modulo fissa la forma di tutte le consegne dell'anno. La prima parte insegna a scrivere i documenti; la seconda insegna a usare l'ambiente in cui d'ora in poi si scriverà il codice. Le due parti si saldano nel punto in cui un notebook alterna celle di codice e celle Markdown: la sintassi studiata nelle prime otto ore è esattamente quella che serve nelle celle di testo del notebook.

Il ruolo della documentazione nel ciclo di vita del software

Un programma non è solo il suo codice. È anche l'insieme delle informazioni che permettono a una persona diversa dall'autore, o allo stesso autore sei mesi dopo, di capire che cosa il programma fa, come lo si installa e quali scelte hanno portato alla struttura che ha. L'assenza di queste informazioni è uno dei motivi più comuni per cui un progetto tecnicamente valido diventa inutilizzabile.

Nel ciclo di vita del software la documentazione cambia destinatario a ogni fase: nell'analisi fissa i requisiti condivisi fra chi commissiona e chi realizza, nella progettazione descrive la struttura scelta e le alternative scartate, nello sviluppo prende la forma dei commenti e delle

istruzioni di compilazione, nel collaudo elenca i casi di prova, nella manutenzione registra le modifiche rilasciate.

Da qui discendono due regole. La prima è che prima di scrivere occorre decidere a chi si scrive: le istruzioni per eseguire un programma devono arrivare al dettaglio del comando da digitare, la spiegazione di un algoritmo può dare per acquisito il linguaggio. La seconda è che la documentazione va scritta insieme al codice, non dopo, perché quella prodotta a distanza di settimane è quasi sempre incompleta: le decisioni piccole — il motivo per cui un valore vale 24 e non 25 — sono già state dimenticate. Il modo più efficace per rispettarla è tenere la documentazione nello stesso repository del codice, così che la modifica al programma e quella al documento entrino nello stesso `commit`. È ciò che Markdown rende possibile.

Che cos'è Markdown e perché si versiona bene

Markdown è un **linguaggio di marcatura leggero**: un insieme di convenzioni per indicare la struttura di un testo — che cosa è un titolo, che cosa è un elenco, che cosa è codice — usando soltanto caratteri ordinari. Un documento Markdown è un normale file di testo con estensione `.md`, che si apre con qualsiasi editor e si legge senza difficoltà anche senza essere convertito. Nasce nel 2004 con un obiettivo dichiarato che ne spiega tutte le scelte di sintassi: il sorgente deve restare leggibile come testo semplice, senza sembrare marcato.

La differenza rispetto a un documento di videoscrittura è sostanziale. Un file `.docx` è un archivio compresso di dati binari: due versioni successive dello stesso documento, per Git, sono due file diversi e basta. Git può registrarne le versioni, ma non può mostrare che cosa è cambiato fra l'una e l'altra, né può fondere le modifiche di due persone che hanno lavorato in parallelo. Un file `.md` è testo puro: `git diff` mostra riga per riga le parole aggiunte e tolte, la fusione automatica funziona come per il codice sorgente, e la cronologia del documento diventa leggibile quanto quella del programma.

Il confronto con l'HTML chiarisce l'altra faccia della questione. In HTML un titolo di secondo livello si scrive `<h2>Requisiti</h2>`; in Markdown si scrive `## Requisiti`. L'HTML è più espressivo ma verboso; Markdown copre le strutture che servono in un documento tecnico e nient'altro. Ed è il formato dei file `README.md` su GitHub e su GitLab, delle celle di testo dei notebook Jupyter che si useranno da qui alla fine dell'anno e della documentazione di quasi tutte le librerie Python: conoscerne la sintassi non serve solo per questo corso.

Titoli, paragrafi ed enfasi

Un titolo si indica con uno o più cancelletti all'inizio della riga, seguiti da uno spazio. Il numero di cancelletti corrisponde al livello: uno per il titolo del documento, due per le sezioni, tre per le sottosezioni, fino a sei.

```
# Relazione di laboratorio

## Obiettivo della prova

### Materiale utilizzato
```

La convenzione quasi universale è che un documento contenga un solo titolo di primo livello, che ne è il titolo, e che i livelli non vengano saltati: dopo un `##` non si passa direttamente a `####`, perché i generatori di indici automatici costruiscono l'indice da questa gerarchia.

I paragrafi si separano con una riga vuota. Questo è il punto in cui sbagliano quasi tutti all'inizio: due righe consecutive senza riga vuota in mezzo, nel documento reso, diventano un unico paragrafo, perché Markdown considera l'andata a capo singola come uno spazio. Per forzare un'interruzione di riga dentro lo stesso paragrafo si chiude la riga con due spazi, oppure — soluzione più leggibile, perché gli spazi finali sono invisibili — con una barra rovesciata.

```
Prima riga della strofa\
Seconda riga della strofa

Questo è un paragrafo nuovo, separato dal precedente
da una riga vuota. Queste due righe del sorgente
diventano un unico paragrafo.
```

L'enfasi si ottiene racchiudendo il testo fra asterischi: uno per il corsivo, due per il grassetto, tre per entrambi. Al posto degli asterischi si possono usare i trattini bassi, ma conviene scegliere una convenzione e mantenerla.

```
Il metodo *deve* restituire un valore intero.
Il parametro **non può** essere negativo.
```

Vale per il Markdown la stessa raccomandazione che vale per questa dispensa: il grassetto va usato con moderazione, per i termini tecnici al primo uso e per le etichette strutturali. Un testo in cui una parola su dieci è in grassetto non evidenzia nulla, perché l'occhio smette di distinguere.

Elenchi, citazioni e righe orizzontali

Un elenco puntato si scrive con un trattino, un asterisco o un più all'inizio della riga, seguito da uno spazio. Un elenco numerato si scrive con un numero seguito da un punto e uno spazio. I livelli annidati si ottengono indentando di due o quattro spazi.

Passi per consegnare l'esercizio:

1. Aprire il terminale nella cartella del repository.
2. Aggiungere i file modificati:
 1. il sorgente Java;
 2. il file `README.md` aggiornato.
3. Creare il commit e inviarlo al repository remoto.

Nel documento reso i numeri vengono rinumerati automaticamente dal convertitore: se nel sorgente si scrive `1.` su tutte le righe, l'elenco reso sarà comunque numerato 1, 2, 3. È una proprietà utile, perché permette di inserire una voce in mezzo a un elenco lungo senza rinumerare le successive.

Le citazioni si indicano con il segno di maggiore all'inizio della riga e servono per riportare il testo di un messaggio d'errore, un estratto di una specifica o una nota di attenzione.

```
> La compilazione fallisce con il messaggio  
> `error: cannot find symbol` quando la classe non è  
> nel classpath.
```

Una riga orizzontale si ottiene con tre o più trattini su una riga isolata, preceduti e seguiti da una riga vuota; va usata con parsimonia, perché in un testo ben strutturato la separazione la fanno i titoli. Attenzione a un'ambiguità che genera errori frequenti: una riga di soli trattini scritta immediatamente sotto una riga di testo, senza riga vuota in mezzo, non produce una riga orizzontale ma trasforma la riga soprastante in un titolo di secondo livello. È la sintassi Setext dei titoli, prevista fin dall'origine e mantenuta ancora oggi.

Collegamenti, immagini e note

Un collegamento si scrive mettendo fra parentesi quadre il testo visibile e subito dopo, fra parentesi tonde, l'indirizzo. L'indirizzo può essere assoluto oppure relativo a un altro file del repository: la seconda forma è quella da preferire per i rimandi interni, perché continua a funzionare anche quando il repository viene clonato o spostato.

```
La specifica completa è su [CommonMark](https://spec.commonmark.org/).  
Le istruzioni di installazione sono nel file [INSTALL](docs/INSTALL.md).  
Il paragrafo sui requisiti è più sopra: [Requisiti](#requisiti).
```

L'ultima forma è un collegamento interno al documento: l'ancora si ricava dal testo del titolo convertito in minuscolo, con gli spazi sostituiti da trattini e la punteggiatura rimossa, quindi `## Requisiti di sistema` diventa `#requisiti-di-sistema`. La regola varia leggermente fra le implementazioni, per cui i collegamenti interni vanno verificati nell'anteprima prima di consegnare.

Un'immagine si scrive come un collegamento preceduto da un punto esclamativo. Il testo fra parentesi quadre non è il titolo dell'immagine ma il **testo alternativo**, quello che viene letto dai lettori di schermo e che compare se il file non è raggiungibile: va scritto in modo descrittivo, non lasciato vuoto.

```
![Diagramma delle classi del registro voti](img/classi.png)
```

Nelle note a piè di pagina si mette il riferimento nel punto del testo e la definizione in fondo al file. L'identificatore è arbitrario e non occorre che sia un numero, perché il convertitore numera le note nell'ordine in cui compaiono: un'etichetta parlante come `[^storia]` resta leggibile nel sorgente.

```
Il formato è stato standardizzato solo dieci anni dopo[^storia].
```

```
[^storia]: La prima versione di CommonMark è del 2014.
```

Le note sono un'estensione, supportata da GitHub, da Pandoc e dalla maggior parte degli strumenti moderni ma non da tutti: prima di usarle in un documento destinato a un ambiente specifico conviene verificare che quell'ambiente le renda.

Codice in linea e blocchi di codice

Un riferimento breve a un comando, a un nome di variabile, a un nome di file o a un metodo si scrive in linea racchiudendolo fra virgolette inverse. L'accorgimento non è decorativo: distingue il nome tecnico dal testo circostante e impedisce al convertitore di interpretare i caratteri speciali che il nome contiene.

Un blocco di codice si delimita con una riga di tre virgolette inverse prima e una dopo. Sulla riga di apertura, subito dopo le virgolette, si indica il linguaggio: questa indicazione attiva la colorazione sintattica ed è considerata obbligatoria nelle consegne di questo corso.

```
```java
public static double media(int[] voti) {
 int somma = 0;
 for (int v : voti) {
 somma += v;
 }
 return (double) somma / voti.length;
}
```
```

I nomi di linguaggio più usati in questo corso sono `java`, `python`, `bash`, `json`, `csv`, `mark-down` e `text` per il testo semplice, per esempio l'output di un programma. Esiste anche la forma indentata, ereditata dal Markdown originale, in cui quattro spazi a inizio riga producono un

blocco di codice: si incontra in documenti vecchi, ma non va usata, perché non permette di indicare il linguaggio.

Mostrare un blocco di codice dentro un blocco di codice

Il problema si presenta ogni volta che si documenta il Markdown stesso, come in questa dispensa. La regola generale è che un **recinto di codice** si chiude solo con una sequenza di virgolette inverse lunga almeno quanto quella di apertura, quindi qualsiasi cosa contenga tre virgolette inverse passa indenne dentro un recinto di quattro.

```
````markdown
```java
System.out.println("Ciao");
```
````
```

Nell'esempio il recinto più esterno è di cinque virgolette inverse, perché deve contenerne uno di quattro, che a sua volta ne contiene uno di tre. È lo stesso meccanismo della protezione delle virgolette dentro una stringa Java: se il contenuto include il carattere che lo delimita, occorre un modo per distinguerlo.

Tabelle

Una tabella si scrive separando le celle con la barra verticale. La seconda riga, obbligatoria, contiene i trattini che separano l'intestazione dal corpo e definisce l'allineamento delle colonne: due punti a sinistra per allineare a sinistra, a destra per allineare a destra, da entrambi i lati per centrare.

```
Comando	Effetto	Frequenza d'uso
`git status`	Mostra lo stato della cartella di lavoro	alta
`git add`	Prepara i file per il commit	alta
`git log --oneline`	Mostra la cronologia in forma compatta	media
```

Le barre verticali del sorgente non devono essere allineate fra loro perché la tabella funzioni, ma allinearle rende il sorgente molto più leggibile e vale la fatica. Due limiti vanno tenuti presenti: le tabelle non fanno parte della specifica di base ma sono un'estensione, e una cella non può contenere un blocco di codice su più righe né un elenco. Se il contenuto è complesso, la tabella non è lo strumento giusto e servono titoli e paragrafi. Per inserire una barra verticale dentro una cella, per esempio per citare l'operatore `||` di Java, la si protegge con una barra rovesciata.

Le varianti: CommonMark e GitHub Flavored Markdown

Il documento originale del 2004 descriveva la sintassi a parole, senza essere una specifica formale: molti casi limite restavano ambigui e implementazioni diverse li risolvevano in modi diversi. **CommonMark** è la risposta a questo problema, una specifica formale avviata nel 2014 e corredata di oltre seicento casi di prova; la versione corrente è la 0.31.2, del 28 gennaio 2024. Non aggiunge costrutti: ne fissa il comportamento.

GitHub Flavored Markdown, abbreviato in GFM, è la variante usata da GitHub ed è un soprainsieme rigoroso di CommonMark: tutto ciò che vale in CommonMark vale in GFM, con cinque aggiunte. Le tabelle, già descritte; le caselle di spunta, cioè voci di elenco che iniziano con `[ ]` o `[x]` e diventano quadratini vuoti o spuntati; il testo barrato, delimitato da due tilde; i collegamenti automatici, per cui un indirizzo scritto per esteso diventa cliccabile senza parentesi; il filtraggio dell'HTML potenzialmente pericoloso, che rimuove marcatori come `<script>`.

Stato dei requisiti:

- `[x]` Lettura del file dei voti
- `[ ]` Esportazione in CSV

La versione `~1.0~ 1.1` corregge l'arrotondamento.

Il repository è su `https://github.com/` e non richiede autenticazione.

La conseguenza pratica è una sola, ed è quella che conta: la stessa sintassi può comportarsi in modo diverso a seconda di dove il documento verrà letto. Le tabelle, le caselle di spunta, il barrato e le note possono non essere rese da uno strumento che implementa solo CommonMark. La regola operativa per questo corso è quindi semplice: si scrive per GitHub Flavored Markdown, che è l'ambiente in cui le consegne vengono lette, e si controlla il risultato nell'anteprima prima di ogni `commit`.

Il file README.md

Il `README.md` è il primo file che si legge quando si apre un repository, e su GitHub è anche l'unico che viene mostrato automaticamente sotto l'elenco dei file. GitHub lo cerca nella radice del repository, oppure nelle cartelle `.github` o `docs`.

Un `README` efficace risponde, nell'ordine, a poche domande precise. Che cosa fa questo programma, in una o due frasi comprensibili anche a chi non conosce il progetto. Che cosa occorre avere installato per eseguirlo. Come si installa e come si avvia, con i comandi esatti da digitare. Come si usa, con almeno un esempio completo di ingresso e di uscita. Come è organizzato il codice. Chi lo ha scritto e con quale licenza è distribuito.

L'errore più comune non è la mancanza di informazioni ma il loro ordine. Un `README` che comincia con la storia del progetto e mette i comandi di installazione in fondo costringe il lettore

a cercare, e il lettore che cerca abbandona: le informazioni vanno ordinate secondo l'urgenza per chi arriva la prima volta, non secondo la cronologia con cui l'autore le ha prodotte. Il secondo errore è il `README` che descrive il programma che si vorrebbe avere anziché quello che c'è. Se una funzione non è implementata va dichiarata tale, e un documento che promette più di quanto il codice mantiene fa perdere più tempo di un documento assente.

Un esempio completo

Segue un `README.md` per un progetto Java del tipo realizzato al terzo anno.

Media Voti

Programma da riga di comando che legge un file CSV di voti scolastici e calcola media, voto minimo e voto massimo per ogni studente. Realizzato come esercitazione di laboratorio della classe 3Bi.

Requisiti

| Componente | Versione minima | Verifica |
|-------------------|--------------------------|------------------------------|
| JDK | 17 | <code>`java -version`</code> |
| Sistema operativo | Windows 10, Linux, macOS | — |

Installazione

1. Clonare il repository:

```
```bash
git clone https://github.com/utente/media-voti.git
cd media-voti
```
```

2. Compilare i sorgenti nella cartella `bin`:

```
```bash
javac -d bin src/*.java
```
```

Uso

Il programma riceve come unico argomento il percorso del file dei voti:

```
```bash
java -cp bin MediaVoti dati/voti.csv
```
```

Formato del file di ingresso

Il file deve essere in formato CSV, con separatore punto e virgola, codifica UTF-8 e una riga di intestazione:

```
```csv
cognome;nome;materia;voto
Rossi;Anna;Informatica;7.5
Rossi;Anna;Matematica;6
Bianchi;Luca;Informatica;8
```
```

I voti sono numeri decimali fra 1 e 10, con il punto come separatore decimale. Le righe con voto fuori intervallo vengono ignorate e segnalate sullo standard error.

Esempio di esecuzione

```
```text
Studente Media Min Max
```

```

Bianchi Luca 8.00 8.0 8.0
Rossi Anna 6.75 6.0 7.5
```

## Struttura del progetto

Percorso	Contenuto
`src/`	Codice sorgente Java
`dati/`	File CSV di esempio
`bin/`	Classi compilate, non versionate

## Limitazioni note

Il programma carica l'intero file in memoria e non è adatto a file di grandi dimensioni. Non gestisce i voti espressi in lettere.

## Autore e licenza

Anna Rossi, classe 3Bi, anno scolastico 2025-2026.
Distribuito con licenza MIT, si veda il file `LICENSE`.

```

Nel documento reso, i blocchi di codice indentati di tre spazi dentro le voci numerate restano associati alla voce dell'elenco anziché interromperlo: è il motivo dell'indentazione, che va conservata quando si copia l'esempio.

La documentazione di un progetto oltre il README

Quando il progetto cresce, la documentazione si articola in più file con ruoli fissati per convenzione. Il file `LICENSE` contiene il testo della licenza e va inserito anche nei progetti scolastici, perché senza licenza esplicita il codice pubblicato resta protetto dal diritto d'autore e formalmente non è riutilizzabile da nessuno. Il file `CONTRIBUTING.md` descrive come proporre modifiche, il file `CHANGELOG.md` elenca le modifiche versione per versione dalla più recente alla più vecchia, la cartella `docs/` raccoglie i documenti lunghi.

Le versioni seguono di norma il **versionamento semantico** nella forma `maggiore.minore.correttiva`: si incrementa il primo numero quando la modifica rompe la compatibilità, il secondo quando aggiunge funzionalità compatibili, il terzo quando corregge soltanto difetti.

```
# Registro delle modifiche

## [1.1.0] - 2026-10-05

### Aggiunto
- Esportazione dei risultati in formato CSV.

### Corretto
- L'arrotondamento della media troncava invece di arrotondare.

## [1.0.0] - 2026-09-28

Prima versione consegnata.
```

Diagrammi testuali con Mermaid

Un diagramma inserito in un documento come immagine ha due difetti: non è modificabile senza lo strumento con cui è stato prodotto, e per Git è un file binario, quindi non se ne può leggere la storia. Mermaid risolve entrambi i problemi permettendo di descrivere il diagramma con del testo, che il visualizzatore trasforma in figura. Si scrive in un blocco di codice con linguaggio `mermaid`: GitHub lo rende automaticamente, Visual Studio Code richiede un'estensione.

Diagramma di flusso

Il diagramma di flusso descrive una sequenza di passi con le sue decisioni. Si apre con la parola chiave `flowchart` seguita dalla direzione: `TD` dall'alto verso il basso, `LR` da sinistra a destra. Ogni nodo ha un identificatore breve e un'etichetta; la forma delle parentesi determina la forma del nodo, con le quadre per i passi e le graffe per le decisioni. Le frecce si scrivono con `→` e possono portare un'etichetta fra barre verticali.

```
```mermaid
flowchart TD
 A[Modifica del codice] → B[Aggiornamento del README]
 B → C{Anteprima corretta?}
 C →|no| B
 C →|sì| D[git add e git commit]
 D → E[git push]
 E → F{Il repository remoto mostra le modifiche?}
 F →|no| G[Verifica del ramo e del remoto]
 G → E
 F →|sì| H[Consegna completata]
```
```

Il risultato reso è uno schema con i rettangoli dei passi, i rombi delle due decisioni e le frecce etichettate che riportano indietro quando la verifica fallisce. Le etichette che contengono virgole, parentesi o due punti vanno racchiuse fra virgolette, per esempio `A["Passo 1: preparazione"]`.

Diagramma delle classi

Il diagramma delle classi descrive la struttura statica di un programma a oggetti. Ogni classe si dichiara con `class` seguito dal nome e da un blocco fra graffe con attributi e metodi. I simboli `+` e `-` indicano visibilità pubblica e privata, come i modificatori `public` e `private` di Java, e le relazioni si indicano con frecce diverse: `*--` per la composizione, `o--` per l'aggregazione, `→` per una semplice associazione. Il diagramma che segue descrive il progetto `media-voti` dell'esempio precedente e usa solo la composizione, perché l'ereditarietà non è ancora stata affrontata.

```

```mermaid
classDiagram
 class Registro {
 -String nomeClasse
 +aggiungiStudente(Studente s)
 +mediaGenerale() double
 }
 class Studente {
 -String cognome
 -String nome
 +media() double
 }
 class Voto {
 -String materia
 -double valore
 +isValido() boolean
 }
 Registro "1" *-- "0..*" Studente : contiene
 Studente "1" *-- "0..*" Voto : registra
```

```

Il risultato reso è composto da tre riquadri divisi in due scomparti, con gli attributi sopra e i metodi sotto, collegati da linee con il rombo pieno della composizione; le stringhe fra virgolette accanto alle estremità sono le molteplicità, e il testo dopo i due punti è il nome della relazione. Mermaid prevede altri tipi di diagramma, fra cui quelli di sequenza per lo scambio di messaggi; la sintassi si trova sul sito ufficiale, che mette a disposizione anche un editor in linea. Quell'editor è un servizio esterno, quindi non vi si incollano dati personali o codice riservato.

Anteprima, conversione ed esportazione

Visual Studio Code integra un'anteprima Markdown senza bisogno di installare nulla: con il file `.md` aperto, `Ctrl+Shift+V` la apre in una scheda e `Ctrl+K` seguito da `V` in una colonna accanto al sorgente, con lo scorrimento sincronizzato. L'anteprima non rende i diagrammi Mermaid, per i quali occorre l'estensione *Markdown Preview Mermaid Support*, e mostra comunque il documento come lo interpreta l'editor, non come lo interpreterà GitHub.

Pandoc è un convertitore da riga di comando che trasforma documenti fra decine di formati ed è lo strumento che genera i PDF di questa dispensa. Si installa da `pandoc.org` e si verifica con `pandoc --version`. L'opzione `-o` indica il file di uscita; `-s`, abbreviazione di *standalone*, è quasi sempre necessaria per l'HTML, perché senza di essa Pandoc produce solo il frammento di corpo del documento; `--toc` genera l'indice, `--embed-resources` incorpora immagini e fogli di stile in un unico file trasportabile, `-f gfm` dichiara che il formato di partenza è quello di GitHub.

```
pandoc -f gfm relazione.md -s --toc --toc-depth=2 --embed-resources \
--metadata title="Relazione di laboratorio" -o relazione.html
```

La conversione in PDF richiede un motore esterno: quello predefinito è `pdflatex`, che fa parte di una distribuzione LaTeX come MiKTeX o TinyTeX. Per i documenti in italiano è preferibile `xelatex`, che gestisce meglio le lettere accentate; le opzioni di impaginazione si passano come variabili con `-V`.

```
pandoc relazione.md -o relazione.pdf --pdf-engine=xelatex \
-V lang=it -V geometry:margin=2.5cm -V fontsize=11pt
```

I diagrammi Mermaid non vengono resi da Pandoc senza aiuto: restano blocchi di codice, e per ottenerli come figure in un PDF si convertono prima in immagini, tenendo però nel repository anche il sorgente del diagramma. Per un PDF rapido, senza installare LaTeX, l'estensione *Markdown PDF* di Visual Studio Code esporta il documento attraverso il motore di rendering dell'editor: è comoda per le bozze, mentre per le consegne il risultato di Pandoc è più controllabile.

Markdown e Git lavorano insieme

Tenere la documentazione in Markdown dentro il repository produce tre effetti. Il primo è che la storia del documento diventa leggibile: `git diff README.md` mostra le righe aggiunte e rimosse fra due versioni, `git log -p README.md` ripercorre le modifiche del file nel tempo. Su un file binario di videoscrittura nessuno dei due comandi produrrebbe qualcosa di utile.

```
git diff README.md
git log --oneline -- README.md
```

Il secondo è che documento e codice restano allineati, perché la modifica al programma e quella alle istruzioni d'uso entrano nello stesso `commit`. Il terzo è che il `README` diventa il punto d'ingresso del repository: chi apre il progetto su GitHub lo vede reso sotto l'elenco dei file ed è spesso l'unica cosa su cui basa il giudizio.

Nel file `.gitignore` non vanno mai messi i file `.md` della documentazione, ma vanno messi i prodotti della conversione, cioè i file `.html` e `.pdf` generati da Pandoc: sono file derivati, ricostruibili dal sorgente, e versionarli riempie la cronologia di modifiche che nessuno leggerà.

```
bin/  
*.class  
*.pdf  
*.html
```

Convenzioni di scrittura tecnica

La sintassi è la parte facile. La parte difficile è scrivere in modo che il lettore capisca al primo passaggio, e i criteri che seguono sono quelli con cui i documenti di questo corso saranno corretti.

Le frasi vanno tenute brevi e con una sola idea ciascuna, e la forma attiva è preferibile alla passiva perché rende esplicito chi compie l'azione: «il programma legge il file di configurazione» dice più di «il file di configurazione viene letto». L'imperativo è la forma corretta per le istruzioni, mentre il condizionale e le formule di cortesia allungano il testo senza aggiungere informazione. La terminologia va usata in modo coerente: se una cosa si chiama `voto`, si chiama `voto` in tutto il documento, perché la variazione lessicale, che in un testo letterario è un pregio, qui fa chiedere al lettore se i termini indichino cose diverse. I nomi tecnici vanno sempre fra virgolette inverse e i comandi riportati esattamente come si digitano.

L'ultimo criterio è il più importante: ogni affermazione verificabile va verificata. Se il documento dice che il programma richiede Java 17, quel numero deve essere stato controllato. La documentazione tecnica perde valore alla prima affermazione falsa che il lettore scopre, perché da quel momento dovrà verificare anche tutte le altre.

L'ambiente Jupyter: interfaccia, server e kernel

Da qui in avanti cambia argomento. Le prossime quattro ore introducono l'ambiente in cui si svolgeranno tutte le lezioni dell'anno, e lo fanno prima di aver scritto una riga di Python: il codice che compare negli esempi serve solo a rendere visibile il comportamento dell'ambiente, non è ancora oggetto di studio.

Un **notebook** è un documento che contiene insieme il codice, i risultati che quel codice ha prodotto e il testo che li spiega. Si salva in un file con estensione `.ipynb` e si apre nel browser. Chi lo usa per la prima volta ha l'impressione di lavorare dentro una pagina web che esegue Python: è una descrizione comoda ma sbagliata, e l'errore si paga la prima volta che qualcosa non funziona. I componenti in gioco sono tre e stanno in posti diversi.

L'**interfaccia** è la pagina che si vede nel browser, cioè JupyterLab. Non esegue codice: mostra le celle, raccoglie quello che si scrive, invia le richieste e disegna le risposte. Chiudere la scheda del browser non ferma nulla di quello che sta girando.

Il **server Jupyter** è un processo che tiene i file `.ipynb` su disco, avvia e ferma i kernel e fa da tramite fra l'interfaccia e i kernel, esponendo un servizio HTTP a cui il browser si collega.

Il **kernel** è un processo Python separato, avviato dal server, che riceve il codice di una cella, lo esegue e restituisce il risultato. Le variabili definite in una cella vivono nella memoria del kernel: non stanno nel file, non stanno nel browser, stanno in quel processo. Quando il kernel viene riavviato tutte le variabili spariscono, anche se i valori che avevano restano stampati sotto le celle, e questa discrepanza fra ciò che si vede e ciò che esiste è la sorgente di gran parte degli errori che si incontreranno quest'anno.

Eseguire una cella significa quindi percorrere una catena: l'interfaccia invia il testo al server, il server lo consegna al kernel, il kernel lo esegue e rimanda indietro il risultato, che risale fino alla pagina. Tre processi, e nessuno di essi è il browser che esegue Python.

La distinzione spiega le voci del menu `Kernel`. `Interrupt` invia il segnale di interruzione e ferma un calcolo che non finisce, senza perdere le variabili. `Restart` termina il processo e ne avvia uno nuovo, vuoto. `Restart Kernel and Run All Cells` riavvia e riesegue tutto dall'inizio, ed è l'operazione con cui si verifica che un notebook funzioni davvero.

Il modello di esecuzione a celle

Un notebook è una sequenza ordinata di **celle**. Le celle di codice contengono Python e producono un output; le celle di testo contengono Markdown e producono testo formattato. Una cella si esegue con `Maiusc+Invio`, che esegue e passa alla successiva, oppure con `Ctrl+Invio`, che esegue e resta ferma. Il risultato compare subito sotto la cella e, se l'ultima riga è un'espressione, il suo valore viene mostrato senza bisogno di stamparlo.

```
ore_settimanali = 4
settimane = 33
ore_settimanali * settimane
```

La cella mostra `132`. Alla sua sinistra compare un numero fra parentesi quadre, per esempio `[1]`: è il **numero di esecuzione**, e dice in che ordine le celle sono state eseguite dall'ultimo avvio del kernel, non in che ordine sono scritte nella pagina. Un `[*]` segnala che la cella è in corso, e finché resta tale nessuna altra cella può partire, perché il kernel esegue una richiesta alla volta; una cella mai eseguita ha le parentesi vuote, `[ ]`.

La proprietà da capire fino in fondo è che lo stato sopravvive alla cella: ogni esecuzione modifica la memoria del kernel, e la modifica resta. Bastano due celle per vederlo.

```
presenti = 22
```

```
print(presenti)
```

Eseguendo la prima e poi la seconda si ottiene `22`, come ci si aspetta. Ora si torni sulla prima cella e si cambi il valore in `20`, **senza rieseguirlo**, e si esegua di nuovo soltanto la seconda: continua a stampare `22`. Il testo della prima cella dice 20, il kernel ricorda 22, e ha ragione il kernel, perché il codice modificato non gli è mai stato inviato. Chi legge il notebook a schermo vede un documento che sembra coerente e non lo è.

L'ordine si può anche invertire. Eseguendo per prima la cella che stampa, quando la variabile non è ancora stata creata, il kernel risponde con un messaggio d'errore:

```
NameError: name 'presenti' is not defined
```

Il nome `presenti` non esiste nella memoria del kernel: non è un difetto del codice, che è corretto, ma dell'ordine in cui le celle sono state eseguite, e i numeri fra parentesi quadre lo raccontano. Se la cella che stampa porta `[5]` e quella che assegna porta `[6]`, la seconda è stata eseguita dopo la prima.

La differenza rispetto a un programma lanciato dal terminale è netta. Un file di codice eseguito da riga di comando parte sempre da uno stato vuoto e percorre le istruzioni nell'ordine in cui sono scritte; un notebook parte da uno stato qualsiasi e percorre le celle nell'ordine in cui l'utente le ha cliccate. La comodità di rieseguire una sola cella senza rifare tutto è il pregio principale dello strumento ed è anche il suo difetto strutturale. Chi arriva da Java riconosce una differenza di modello, non di sintassi: il ciclo `compila ed esegui` produce sempre lo stesso risultato a parità di sorgente, mentre in un notebook il sorgente è solo una parte di ciò che determina il risultato.

Un ultimo esempio, con un costrutto che si studierà fra due settimane e che qui serve solo a mostrare un output su più righe:

```
for numero in range(1, 4):  
    print("riga", numero)
```

Celle di codice e celle Markdown

Le celle di testo di un notebook si scrivono nel Markdown della prima parte di questo modulo: titoli con i cancelletti, elenchi, tabelle, enfasi, collegamenti, codice in linea fra virgolette inverse. Il tipo della cella si cambia dal menu a tendina della barra degli strumenti, oppure premendo `Esc` seguito da `M` per passare a Markdown e `Esc` seguito da `Y` per tornare a codi-

ce. Una cella Markdown si «esegue» come una cella di codice, con `Maiusc+Invio`: l'esecuzione ne mostra il testo reso al posto del sorgente.

Valgono due indicazioni aggiuntive. La prima è che i titoli alimentano il sommario laterale di JupyterLab, quindi vanno usati con la gerarchia corretta e non scelti in base alla dimensione del carattere che producono. La seconda è che una cella di testo non deve descrivere che cosa fa il codice sottostante, che si legge da sé, ma perché quel codice è lì: quale domanda sta cercando di risolvere, quale scelta è stata fatta, come va letto il risultato che segue.

È il punto in cui le due metà di questo modulo si saldano. Un notebook consegnato senza celle di testo non è un notebook, è un programma scritto in modo scomodo; e le celle di testo si valutano con gli stessi criteri di scrittura tecnica visti sopra, perché sono lo stesso oggetto.

I comandi magici

Dentro una cella di codice si possono scrivere alcune istruzioni che non fanno parte di Python ma sono interpretate dal kernel: si chiamano **comandi magici** e iniziano con `%` se valgono per una riga, con `%%` se valgono per l'intera cella.

Quelli che servono da subito sono tre. `%who` elenca le variabili definite nel kernel, ed è lo strumento diagnostico naturale per il problema descritto nella sezione precedente. `%%time`, scritto sulla prima riga di una cella, ne misura il tempo di esecuzione. `%reset -f` svuota la memoria del kernel senza riavviarlo.

```
presenti = 22
assenti = 3
%who
```

Un punto esclamativo all'inizio della riga esegue invece un comando del sistema operativo, per esempio `!ls`. Va usato con parsimonia e mai per installare librerie dentro un notebook da consegnare, che diventerebbe non riproducibile perché il suo effetto dipenderebbe dallo stato della macchina.

Il Jupyter server d'istituto

Il server su cui si lavora non gira sul PC della postazione: gira sul server d'istituto, quello già incontrato nel modulo M0. L'accesso avviene dal browser, all'indirizzo comunicato a lezione, con le credenziali d'istituto; ogni studente lavora in una propria cartella personale e non vede quelle degli altri.

Ne discendono due conseguenze pratiche. La prima è che i file del notebook restano sul server e non nella cartella `Documenti` della postazione: per portarseli via si usa la voce `Download` del menu contestuale dei file, oppure si sincronizza la cartella con Git, che è la strada

adottata in questo corso. La seconda è che il codice viene eseguito dai processori del server, quindi un calcolo pesante non rallenta la postazione ma la rallenta per tutti gli altri che stanno usando il server.

La sequenza di lavoro di una lezione tipo è la seguente.

1. Collegarsi dal browser all'indirizzo del server, autenticandosi con le credenziali d'istituto.
2. Aprire la propria cartella personale e creare un nuovo notebook con `File > New > Notebook`, scegliendo il kernel `Python 3`.
3. Rinominare subito il file con un nome descrittivo, per esempio `M1_prime_celle.ipynb`: il nome predefinito `Untitled.ipynb` diventa ingestibile dopo tre lezioni.
4. Salvare spesso con `Ctrl+S`, senza affidarsi al salvataggio automatico.
5. Prima di chiudere, eseguire `Kernel > Restart Kernel and Run All Cells` e controllare che il notebook arrivi in fondo senza errori.
6. Fermare il kernel con `File > Shut Down` oppure dalla sezione dei kernel attivi di Jupyter-Lab.

L'ultimo passo va rispettato: un kernel dimenticato aperto continua a occupare memoria sul server anche a lezione finita, e ventiquattro kernel dimenticati sono un problema serio. Vale inoltre una regola sui dati: sul server non si caricano dati personali di terzi, come fotografie, elenchi di nominativi reali o esportazioni del registro elettronico.

Riproducibilità di un notebook

Un lavoro è **riproducibile** quando un'altra persona, con lo stesso materiale, ottiene lo stesso risultato. Su un notebook la riproducibilità non è garantita da niente, per le ragioni viste nel modello di esecuzione, e va costruita.

L'unica verifica valida prima di consegnare è `Kernel > Restart Kernel and Run All Cells`. Se il notebook arriva in fondo senza errori, quello che si vede corrisponde davvero al codice scritto; se si ferma a metà, il notebook non è pronto. Il caso tipico è la cella cancellata: si definisce qualcosa, lo si usa più avanti, poi si cancella la cella che lo definiva senza accorgersene, perché quel qualcosa continua a esistere nella memoria del kernel. Il notebook funziona sullo schermo e non funziona per nessun altro. La regola del corso è quindi una sola frase, e vale da qui fino a giugno: **prima di consegnare, riavvia il kernel ed esegui tutto dall'inizio**.

Un secondo accorgimento riguarda i file letti dal notebook. Un notebook che legge `C:\Users\marco\Desktop\dati.csv` non funziona su nessun'altra postazione, e in particolare non sul server, dove quel percorso non esiste. I file di dati vanno tenuti nella stessa cartella del notebook e richiamati con un percorso relativo, come i collegamenti relativi di un documento Markdown.

Il notebook e Git

Il file `.ipynb` è un file di testo in formato JSON. Contiene l'elenco delle celle e, per ciascuna, il tipo, il testo sorgente riga per riga, il numero di esecuzione e tutti gli output prodotti, immagini comprese; queste ultime sono codificate come lunghissime stringhe di caratteri, tanto che una figura da 100 KB occupa circa 140 000 caratteri dentro il file. Ecco una sola cella di codice, così come Git la vede:

```
{
  "cell_type": "code",
  "execution_count": 7,
  "id": "a1b2c3d4",
  "metadata": {},
  "outputs": [
    { "name": "stdout", "output_type": "stream", "text": ["presenti: 22\n"] }
  ],
  "source": ["print('presenti:', presenti)"]
}
```

La riga di codice utile è una, le righe versionate sono una dozzina, e alcune cambiano a ogni riesecuzione anche senza toccare il codice: `execution_count` diventa `8`, `text` cambia se il dato è cambiato, e se la cella produce un'immagine l'intero blocco codificato viene riscritto da capo.

La conseguenza è che `git diff` su un notebook è illeggibile: la modifica di una parola in un commento può comparire dentro un `diff` di quattromila righe, in mezzo a blocchi che nessuno può confrontare a occhio. La revisione del lavoro altrui diventa impraticabile e la cronologia del repository perde il valore documentale che il modulo M0 le attribuiva.

La contromisura è pulire gli output prima del commit. Dall'interfaccia si fa con `Kernel > Restart Kernel and Clear Outputs of All Cells`; da terminale con una riga:

```
jupyter nbconvert --clear-output --inplace primo_notebook.ipynb
```

Il file resta un notebook completo e funzionante, ma senza risultati salvati, e il `diff` mostra soltanto le righe effettivamente cambiate. Il prezzo è che chi apre il notebook su GitHub non vede i risultati finché non lo esegue: è accettabile per il lavoro in corso, mentre per una consegna finale si può fare un commit con gli output e dichiararlo nel messaggio.

Vale anche qui la regola sui file derivati, per cui nel `.gitignore` del repository personale vanno aggiunte le righe seguenti, dove `.ipynb_checkpoints/` è la cartella dei salvataggi automatici di Jupyter.

```
.ipynb_checkpoints/  
dati/  
figure/
```

Da qui in avanti l'ambiente non verrà più rispiegato: **dal modulo M2 tutte le lezioni si svolgono su notebook**, eseguiti sul server d'istituto, e ogni consegna dell'anno sarà un notebook versionato nel repository personale accanto ai documenti Markdown.

Errori frequenti

Markdown non ha un interprete che segnala gli errori: un documento sbagliato viene reso comunque, solo in modo diverso da quello voluto. Il primo strumento diagnostico è quindi l'anteprima. Il kernel di Jupyter, al contrario, i messaggi li produce, e vanno letti.

Paragrafi che si fondono. Due righe consecutive senza riga vuota diventano un unico paragrafo, e nell'anteprima si riconosce perché il testo prosegue sulla stessa riga invece di andare a capo.

Titolo o elenco che non si riconoscono. Il cancelletto e il trattino devono stare all'inizio della riga ed essere seguiti da uno spazio: `##Requisiti` resta testo normale. Un elenco vuole inoltre una riga vuota fra il paragrafo precedente e la prima voce.

Blocco di codice mai chiuso. Se manca il recinto di chiusura, tutto il resto del documento viene reso come codice: da un certo punto in poi il testo diventa a spaziatura fissa e su fondo grigio. Si corregge risalendo all'ultimo recinto aperto. Il caso particolare è il recinto interno che chiude quello esterno, mostrando Markdown dentro Markdown: la soluzione è il recinto esterno di quattro virgolette inverse.

Tabella che non diventa tabella. Manca la riga dei trattini sotto l'intestazione, oppure il numero di celle di una riga non corrisponde a quello dell'intestazione. Nell'anteprima si vede il sorgente della tabella, con tutte le barre verticali, reso come testo normale.

Immagine che non compare. Quasi sempre il percorso è sbagliato o è assoluto e riferito al proprio computer. Va controllata anche la differenza fra maiuscole e minuscole: su Windows non conta, sui server di GitHub sì, e un'immagine che si vede in locale può sparire in linea.

Pandoc non produce il PDF. Se il messaggio è `pdflatex not found` manca la distribuzione LaTeX: si installa MiKTeX o TinyTeX, oppure si sceglie un altro motore con `--pdf-engine`. Un errore del tipo `Unicode character ... not set up for use with LaTeX` indica un carattere che il motore predefinito non conosce e si risolve passando a `xelatex`.

Il kernel dice `NameError: name '...' is not defined`. Il nome non esiste nella memoria del kernel: quasi sempre la cella che lo definiva non è stata eseguita, oppure lo è stata prima di un

riavvio. Si controlla con `%who` e si risolve eseguendo le celle nell'ordine giusto, o riavviando ed eseguendo tutto.

La cella resta con `[*]` e non risponde. Il kernel è occupato da un'esecuzione precedente che non termina. Si usa `Kernel > Interrupt`; se non basta, `Kernel > Restart`, perdendo però tutte le variabili.

Il notebook funziona a schermo ma non per il docente. È il difetto tipico dell'esecuzione fuori ordine: i risultati visibili sono stati prodotti da una versione del codice che non c'è più. Si riconosce sempre allo stesso modo, con `Restart Kernel and Run All Cells`.

Sintesi

Markdown è un linguaggio di marcatura leggero che descrive la struttura di un documento con caratteri ordinari, mantenendo il sorgente leggibile. Essendo testo puro si versiona con Git come il codice: se ne possono leggere le differenze, ripercorrere la storia e fondere il lavoro di più persone.

La sintassi copre i titoli con i cancelletti, l'enfasi con gli asterischi, gli elenchi con i trattini o i numeri, le citazioni con il segno di maggiore, i collegamenti con parentesi quadre e tonde, le immagini con la stessa forma preceduta dal punto esclamativo, il codice in linea e i blocchi recintati con l'indicazione del linguaggio, le tabelle con le barre verticali. Per mostrare un blocco dentro un altro si usa un recinto esterno più lungo di quello interno. CommonMark è la specifica formale che fissa il comportamento; GitHub Flavored Markdown la estende con tabelle, caselle di spunta, testo barrato, collegamenti automatici e filtraggio dell'HTML pericoloso, ed è la variante in cui si scrivono le consegne del corso.

Il `README.md` è il punto d'ingresso di un repository e risponde nell'ordine a che cosa fa il programma, che cosa serve per eseguirlo, come si installa, come si usa e come è organizzato. Mermaid rende versionabili anche i diagrammi; l'anteprima verifica il documento durante la scrittura, Pandoc lo converte in HTML e in PDF. La qualità di un documento tecnico non dipende dalla sintassi ma dalla scrittura: frasi brevi, forma attiva, imperativo nelle istruzioni, terminologia coerente e nessuna affermazione non verificata.

L'ambiente Jupyter è fatto di tre componenti: l'interfaccia nel browser, che non esegue nulla; il server, che tiene i file e avvia i kernel; il kernel, un processo separato in cui vivono le variabili. Il numero fra parentesi quadre indica l'ordine di esecuzione e non quello di scrittura, e lo stato del kernel sopravvive alle singole celle: da qui la possibilità che un notebook coerente a schermo non sia riproducibile. La verifica obbligatoria prima di ogni consegna è il riavvio del kernel con esecuzione completa. Il file `.ipynb` è JSON e contiene anche gli output, il che rende `git diff` illeggibile: prima del commit gli output vanno puliti. Dal modulo M2 tutte le lezioni si svolgono su notebook.

Glossario

| Termine italiano | Termine inglese | Significato |
|---------------------------------|-----------------------------|--|
| Linguaggio di marcatura leggero | lightweight markup language | Notazione che descrive la struttura di un testo con caratteri ordinari, mantenendo leggibile il sorgente |
| Testo puro | plain text | File composto di soli caratteri, senza formattazione binaria; è il formato che Git confronta riga per riga |
| Documento reso | rendered document | Risultato della conversione del sorgente Markdown nella forma finale, HTML o PDF |
| Anteprima | preview | Visualizzazione del documento reso aggiornata durante la scrittura |
| Recinto di codice | code fence | Le tre o più virgolette inverse che delimitano un blocco di codice |
| Codice in linea | inline code | Frammento breve di codice inserito nel testo fra virgolette inverse |
| Testo alternativo | alt text | Descrizione di un'immagine, letta dai lettori di schermo e mostrata se l'immagine manca |
| Casella di spunta | task list item | Voce di elenco resa come quadratino spuntabile, estensione di GitHub Flavored Markdown |
| Soprainsieme rigoroso | strict superset | Variante che accetta tutto ciò che accetta la base e in più altri costrutti |
| File derivato | build artifact | File prodotto automaticamente da un sorgente, che non va versionato |
| Versionamento semantico | semantic versioning | Convenzione <code>maggiore.minore.correttiva</code> per numerare le versioni |
| Molteplicità | multiplicity | Numero di istanze coinvolte a un'estremità di una relazione in un diagramma delle classi |
| Composizione | composition | Relazione in cui l'oggetto contenuto non esiste indipendentemente dal contenitore |
| Diagramma di flusso | flowchart | Rappresentazione grafica di una sequenza di passi e decisioni |
| Notebook | notebook | Documento che contiene insieme codice, risultati prodotti e testo di spiegazione |
| Cella | cell | Unità di un notebook, di tipo codice o Markdown, eseguibile singolarmente |
| Kernel | kernel | Processo separato che esegue il codice delle celle e conserva le variabili |

| Termine italiano | Termine inglese | Significato |
|----------------------|-----------------|--|
| Numero di esecuzione | execution count | Numero fra parentesi quadre che indica l'ordine in cui le celle sono state eseguite |
| Comando magico | magic command | Istruzione del kernel, non di Python, introdotta da <code>%</code> o <code>%%</code> |
| Riproducibilità | reproducibility | Proprietà di un lavoro che, con lo stesso materiale, dà a chiunque lo stesso risultato |